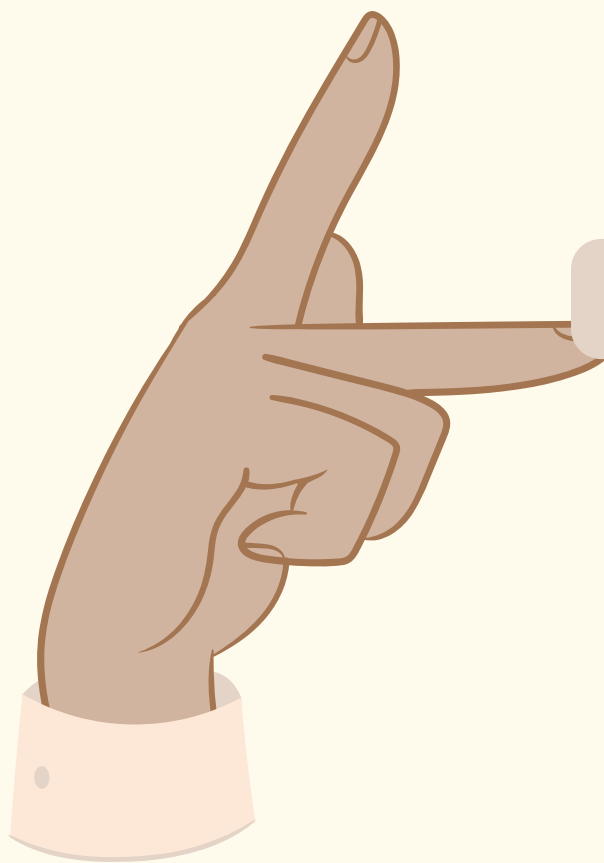




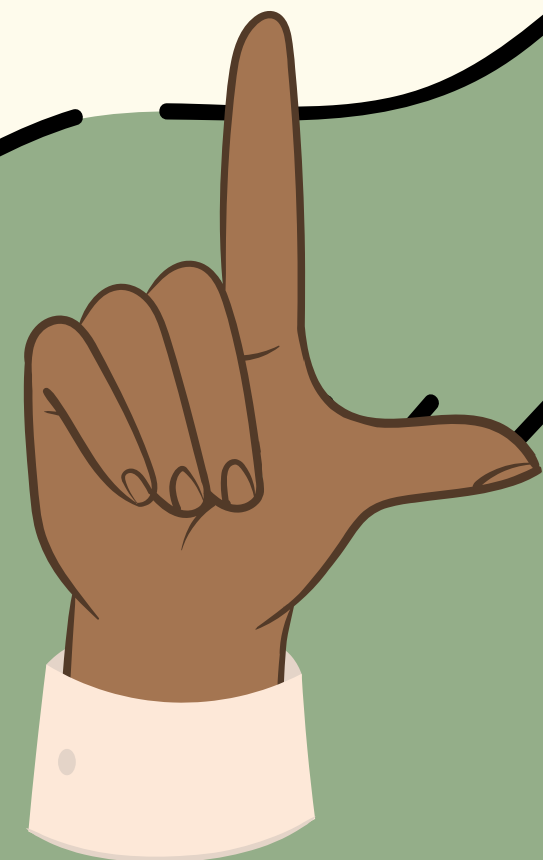
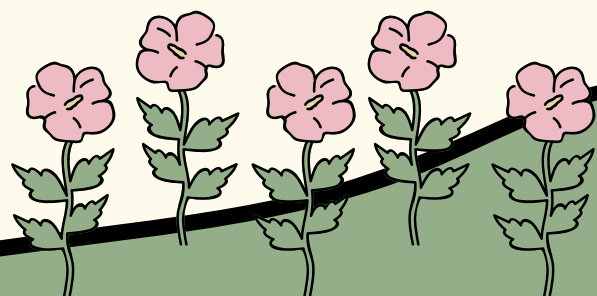
Sign Language

411770281 溫苡均 411770349 黃妤涵 411770547 李佩臻

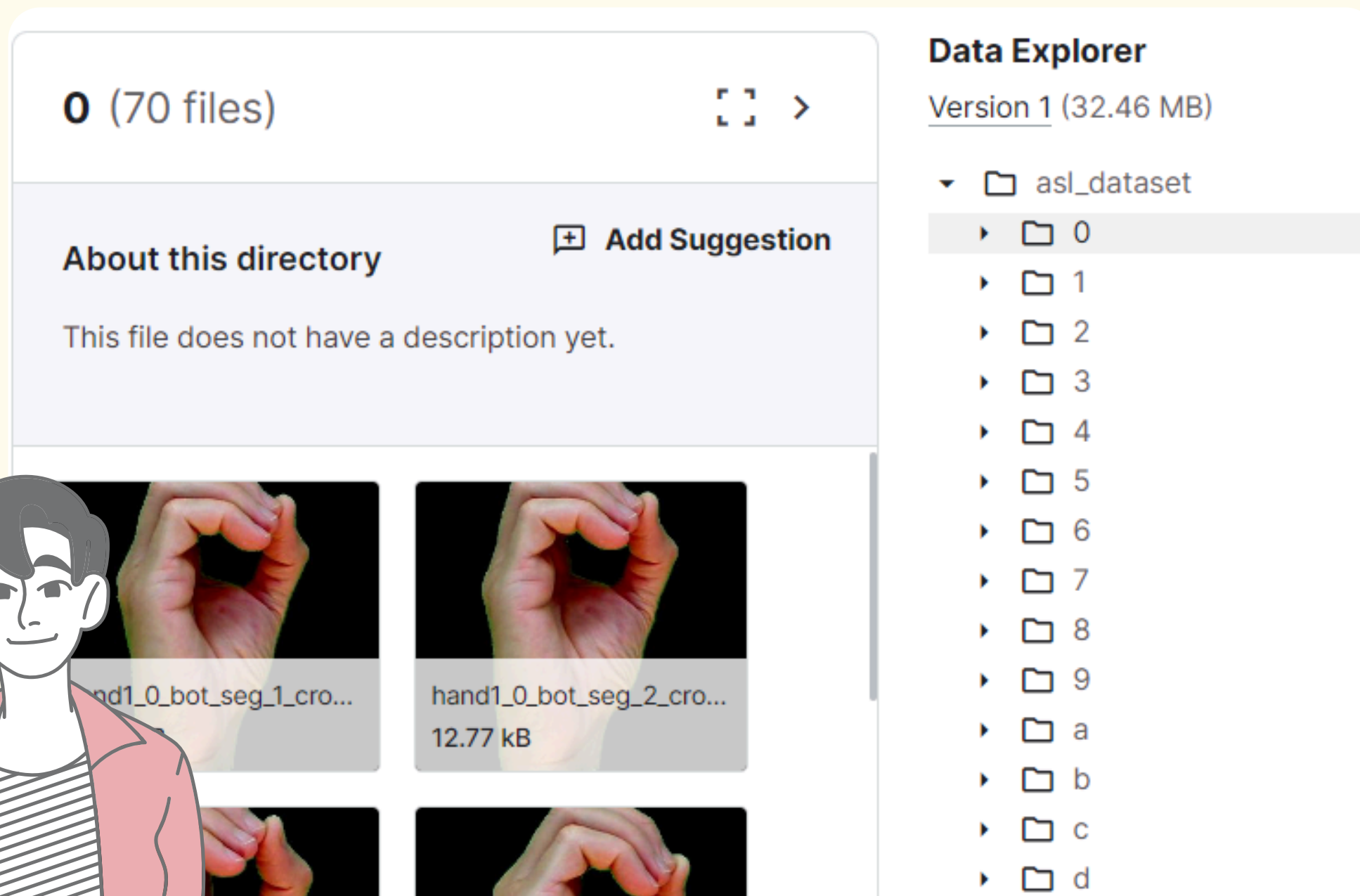


動機與目的

希望能透過所架設網頁輸入手勢圖片，辨識出該手勢的意思，並判斷其正負面意義，以協助不擅長手語的人辨識



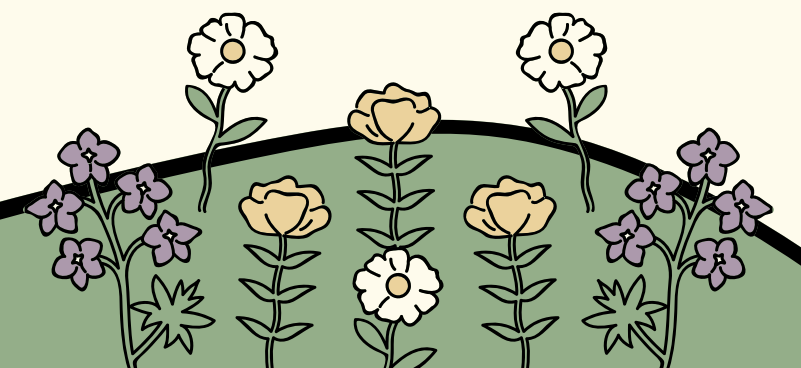
資料集介紹



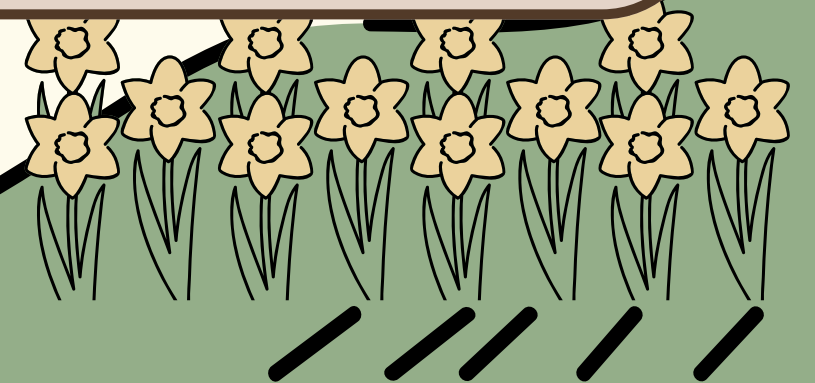
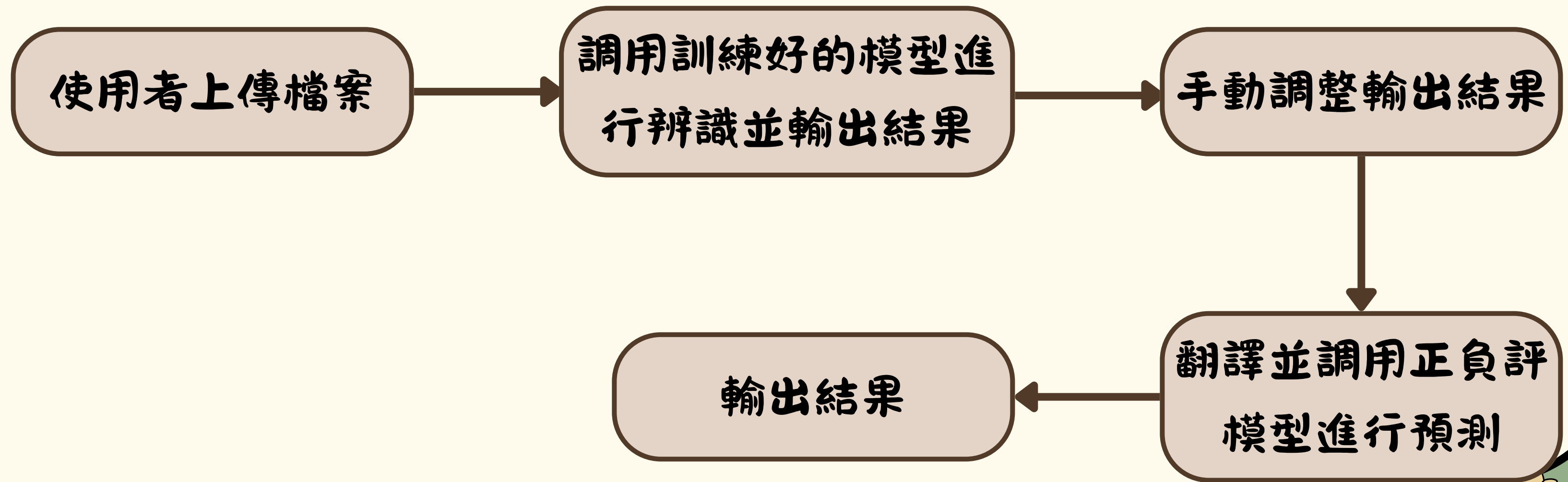
資料集中含有以0-9及a-z
的分類的手語圖片

資料集來源：

<https://www.kaggle.com/datasets/ayuraj/american-sign-language-dataset>



流程架構圖



介面呈現

```
File Edit Selection View Go Run Terminal Help
AI2DOWN2
> asl_dataset
> base
> W11
  > bert_chinese_model
  > static
    > css
      > pic.png
      > # style.css
    > img
    > js
      > JS event.js
  > templates
    > home.html
    > first_model.h5
  > main.py
  > bert_hugging_face.ipynb
  > home_a.html
  > home.html
  > main.py
  > model.ipynb
  > no_number.h5
  > pip.ipynb
  > positive.ipynb
  > requirement.txt
  > sentence_base.h5
  > tensorflow_gpu-2.6.0-cp38-cp3...
  > translate.ipynb

W11 > main.py > upload
35 # 評語文字生成
36 def predict_sentiment(text, model, tokenizer):
37     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
38     model = model.to(device)
39     encodings = preprocess_text(text, tokenizer)
40     with torch.no_grad():
41         inputs = {key: val.to(device) for key, val in encodings.items()}
42         outputs = model(**inputs)
43         prediction = torch.argmax(outputs.logits, dim=-1)
44
45     print(prediction)
46     if prediction.item() == 1:
47         return "正評"
48
49     else:
50         return "負評"
51
52 @app.route("/")
53 def index():
54
55     return render_template('home.html')
56
57 @app.route("/upload", methods=["GET", "POST"])
58 def upload():
59     if request.method == "POST":
60         upload_files = request.files.getlist("file[]")
61         filenames = []
62         for file in upload_files:
63             filename = secure_filename(file.filename)
64
65             filenames.append(filename)
66             file.save(UPLOAD_FOLDER + "\\" + file.filename)
67         return render_template('home.html')
68
69 @app.route("/predict_image", methods=["GET", "POST"])
70 def predict_image():
71     # model = load_model('sentence_base.h5')
72     model = load_model('W11/first_model.h5')
73     result_str = []
74     my_path = "C:\\Users\\c\\OneDrive\\Desktop\\AI2down2\\W11\\static\\img"
```

1/1 [=====] - 0s 14ms/step
1/1 [=====] - 0s 13ms/step
1/1 [=====] - 0s 17ms/step
127.0.0.1 - - [04/Jun/2024 04:03:53] "GET /predict_image HTTP/1.1" 200 -
tensor([0])
127.0.0.1 - - [04/Jun/2024 04:03:57] "POST /predict_text HTTP/1.1" 200 -

Ln 57, Col 45 Spaces: 2 UTF-8 CRLF Python 3.8.19 (net: conda) Port: 5500

23°C
上午 04:05
2024/6/4



模型訓練資料量

- 手語分類模型：

train data: 80%

test data: 20%

共36個類別

2515張照片

- 正負評分分類模型：

擷取2098則評論



程式碼解釋： 手語辨識模型訓練

確認圖片大小



```
1 from PIL import Image
2 img = Image.open(r'C:\Users\clair\Desktop\work\AI2down2\asl_dataset\2\hand1_2_bot_seg_2_cropped.jpeg')
3 print(img.size)
```

分割資料集

```
1 from sklearn.model_selection import train_test_split
2 import shutil
3 #訓練集與測試集分割
4 # 定義原始數據集的路徑
5 original_dataset_dir = r'C:\Users\clair\Desktop\work\AI2down2\asl_dataset'
6
7 # 定義新的目錄結構
8 base_dir = r'C:\Users\clair\Desktop\work\AI2down2\base'
9 train_dir = os.path.join(base_dir, 'train')
10 test_dir = os.path.join(base_dir, 'test')
11 train_categories = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l']
12 test_categories = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l']
13
14 # 建立新的目錄
15 os.makedirs(train_dir, exist_ok=True)
16 os.makedirs(test_dir, exist_ok=True)
17
18 # 將數據集劃分為訓練集和測試集
19 for category in train_categories:
20     os.makedirs(os.path.join(train_dir, category), exist_ok=True)
21     os.makedirs(os.path.join(test_dir, category), exist_ok=True)
22     category_dir = os.path.join(original_dataset_dir, category)
23     image_files = [f for f in os.listdir(category_dir) if os.path.isfile(os.path.join(category_dir, f))]
24     train_images, test_images = train_test_split(image_files, test_size=0.2, random_state=42)
25
26     # 將圖片複製到新的目錄
27     for image in train_images:
28         shutil.copy(os.path.join(category_dir, image), os.path.join(train_dir, category, image))
29     for image in test_images:
30         shutil.copy(os.path.join(category_dir, image), os.path.join(test_dir, category, image))
31
```

模型訓練

```
1 np.random.seed(1)
2
3 label_names=['0','1','2','3','4','5','6','7','8','9','a']
4
5 def load_data_from_folders(data_dir,label_names):
6     images=[]
7     labels=[]
8     for label_id,label in enumerate(label_names):
9         folder_path=os.path.join(data_dir,label)
10        for file_name in os.listdir(folder_path):
11            file_path=os.path.join(folder_path,file_name)
12            img=cv2.imread(file_path)
13            img=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
14            images.append(img)
15            labels.append(label_id)
16    return np.array(images),np.array(labels)
```

```
train_data_dir=r'C:\Users\clair\Desktop\work\AI2down2\base\train'
X_train,Y_train=load_data_from_folders(train_data_dir,label_names)
```

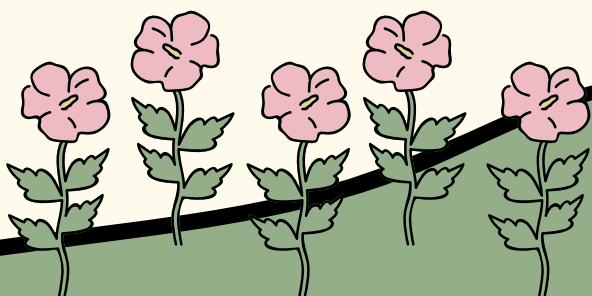
```
test_data_dir=r'C:\Users\clair\Desktop\work\AI2down2\base\test'
X_test,Y_test=load_data_from_folders(test_data_dir,label_names)
```

```
X_train=X_train.astype('float32')/255.0
X_test=X_test.astype('float32')/255.0
Y_train=to_categorical(Y_train,num_classes=len(label_names))
Y_test=to_categorical(Y_test,num_classes=len(label_names))
```

模型訓練

```
model=Sequential([
    Conv2D(32,kernel_size=(3,3),padding='same',activation='relu',input_shape=(400,400,3)),
    MaxPooling2D(pool_size=(2,2)),
    Dropout(0.25),
    Conv2D(64,kernel_size=(3,3),padding='same',activation='relu'),
    MaxPooling2D(pool_size=(2,2)),
    Dropout(0.25),
    Flatten(),
    Dense(512,activation='relu'),
    Dropout(0.5),
    Dense(36,activation='softmax')
])
```

```
from keras.preprocessing.image import ImageDataGenerator
datagen=ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)
datagen.fit(X_train)
```





程式碼解釋： 正負評模型訓練

訓練資料準備

```
1 with open("/kaggle/input/positive-negative/正負面判斷/positive.txt", "r", encoding = "utf-8") as f:
2     positive_words = set(f.read().splitlines())
3
4 with open("/kaggle/input/positive-negative/正負面判斷/negative.txt", "r", encoding = "utf-8") as f:
5     negative_words = set(f.read().splitlines())
6
7 def score_sentence(sentence):
8     words = jieba.lcut(sentence)
9
10    positive_matches = [word for word in words if word in positive_words]
11    negative_matches = [word for word in words if word in negative_words]
12
13    score = len(positive_matches) - len(negative_matches)
14    return score, words, positive_matches, negative_matches
15
16 output_rows = [["Sentence", "Score", "Sentiment"]]
17
18 with open("/kaggle/input/newdata/online_shopping_10_cats.csv", "r", encoding = "utf-8") as file:
19     reader = csv.reader(file)
20     for row in reader:
21         sentence = row[0]
22         score, words, positive_matches, negative_matches = score_sentence(sentence)
23         #print(score, words, positive_matches, negative_matches)
24
25         if score > 0:
26             sentiment = "正評"
27
28         else:
29             sentiment = "負評"
30
```

模型訓練

```
26 # 訓練模型
27 def train_model(model, data_loader, optimizer, device):
28     """在一個訓練週期內訓練BERT模型，計算損失與準確率。"""
29     model.train()
30     total_loss = 0
31     total_correct = 0
32     total_samples = 0
33
34     for batch in data_loader:
35         batch = {k: v.to(device) for k, v in batch.items()} # 將資料移至CPU或GPU
36         outputs = model(**batch) # 模型前向傳播
37         loss = outputs.loss
38         total_loss += loss.item()
39         loss.backward() # 反向傳播計算梯度
40         optimizer.step() # 更新模型參數
41         optimizer.zero_grad() # 清除舊梯度
42
43         # 計算準確率
44         preds = torch.argmax(outputs.logits, dim=-1)
45         total_correct += (preds == batch['labels']).sum().item()
46         total_samples += batch['labels'].size(0)
47
48     # 計算平均損失和準確率
49     avg_loss = total_loss / len(data_loader)
50     accuracy = total_correct / total_samples
51     return avg_loss, accuracy
52
53 # 主程式
54 def main():
55     train_texts, test_texts, train_labels, test_labels = prepare_data(DATA_PATH)
56     tokenizer = BertTokenizer.from_pretrained(MODEL_NAME)
57     train_data = TextDataset(train_texts.tolist(), train_labels.tolist(), tokenizer)
58     train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, num_workers=0)
59
60     model = BertForSequenceClassification.from_pretrained(MODEL_NAME, num_labels=2)
61     model.to(torch.device("cuda" if torch.cuda.is_available() else "cpu")) # 移動模型至GPU中
62     optimizer = AdamW(model.parameters(), lr=2e-5) # 定義優化器
63
64     for epoch in range(EPOCHS):
65         train_loss, train_accuracy = train_model(model, train_loader, optimizer, model.device)
66         print(f'Epoch {epoch + 1}, Train Loss: {train_loss:.4f}, Train Accuracy: {train_accuracy:.4f}')
67
68     model.save_pretrained('bert_chinese_model') # 保存訓練好的模型
69
70 if __name__ == '__main__':
71     main()
72
```

```
1 # 設定參數
2 DATA_PATH = '/kaggle/working/score_comments.csv'
3 MODEL_NAME = 'bert-base-chinese'
4 BATCH_SIZE = 64
5 EPOCHS = 10
```

調用模型預測

```
4 # 載入模型和分詞器
5 def load_model_and_tokenizer(model_path, tokenizer_name):
6     model = BertForSequenceClassification.from_pretrained(model_path)
7     tokenizer = BertTokenizer.from_pretrained(tokenizer_name)
8     model.eval() # 將模型設定為評估模式
9     return model, tokenizer
10
11 # 预处理文本
12 def preprocess_text(text, tokenizer, max_len=128):
13     encodings = tokenizer(text, truncation=True, padding='max_length', max_length=max_len, return_tensors='pt')
14     return encodings
15
16 # 预测文本情感
17 def predict_sentiment(text, model, tokenizer):
18     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
19     model = model.to(device)
20     encodings = preprocess_text(text, tokenizer)
21     with torch.no_grad():
22         inputs = {key: val.to(device) for key, val in encodings.items()}
23         outputs = model(**inputs)
24         prediction = torch.argmax(outputs.logits, dim=-1)
25
26     print(prediction)
27     if prediction.item() == 1:
28         return "正評"
29
30     else:
31         return "負評"
32
33     #else:
34     #return "中立"
35
36 # 主功能
37 def main():
38     model_path = '/kaggle/working/bert_chinese_model'
39     tokenizer_name = 'bert-base-chinese'
40     model, tokenizer = load_model_and_tokenizer(model_path, tokenizer_name)
41     new_text = "憤怒"
42     sentiment = predict_sentiment(new_text, model, tokenizer)
43     print(f"預測的文本情感: {sentiment}")
44
45 if __name__ == '__main__':
46     main()
47
```



程式碼解釋： 前後端連接

```
1 UPLOAD_FOLDER = r'C:\Users\clair\Desktop\AI2down2\W11\static\img'
```

```
@app.route("/")
def index():

    return render_template('home.html')

@app.route("/upload", methods=["GET", "POST"])
def upload():
    if request.method=="POST":
        upload_files=request.files.getlist("file[]")
        filenames=[]
        for file in upload_files:
            filename = secure_filename(file.filename)

            filenames.append(filename)
            file.save(UPLOAD_FOLDER+"\\ "+file.filename)
        return render_template('home.html')
```

flask- main.py

載入套件

```
import numpy as np
import cv2
import os
from os import walk
from os.path import join
import googletrans
from pprint import pprint
from keras.models import load_model
from keras.utils import to_categorical
from keras.models import Sequential
from keras.layers import Dense, Flatten, Conv2D, MaxPooling2D, Dropout
from werkzeug.utils import secure_filename
from flask import Flask
from flask import Flask, render_template, request, redirect, url_for, Response, jsonify
15 import torch
16 from transformers import BertTokenizer, BertForSequenceClassification
```

main.py

```
@app.route("/predict_image",methods=[ "GET",'POST'])
def predict_image():
    # model=load_model('sentence_base.h5')
    model=load_model('W11/first_model.h5')
    result_str=[]
    mypath=r"C:\Users\clair\Desktop\AI2down2\W11\static\img"
    for files in walk(mypath):
        for f in files[2]:

            image_path=join(files[0],f)

            image=cv2.imread(image_path)
            image=cv2.cvtColor(image,cv2.COLOR_BGR2RGB)

            image=cv2.resize(image,(400,400))
            image=image.astype('float32')/255.0
            image=np.expand_dims(image,axis=0)

            predictions=model.predict(image)
```

```
20
21     predicted_class=np.argmax(predictions,axis=1)
22     class_result=predicted_class
23     pre_class=[[0],[1],[2],[3],[4],[5],[6],[7],[8],[9],[10],
24     char=['0','1','2','3','4','5','6','7','8','9','a','b','c
25     # pre_class=[[0],[1],[2],[3],[4],[5],[6],[7],[8]]
26     # char=["a","e","f","l","o","r","s","u","v"]
27     index=pre_class.index(class_result)
28     char_class=char[index]
29
30     result_str.append(char_class)
31 arr=np.array(result_str)
32 str=''.join(arr)
33
34     for files in walk(mypath):
35         for f in files[2]:
36             image_path=join(files[0],f)
37             os.remove(image_path)
38
39     return jsonify({"result":str})
```

js(圖片預測)

```
1 function post_data_image() {
2
3
4     $.ajax({
5         url: "/predict_image",
6         type: "GET",
7         dataType: "json",
8         processData: false, //用於對data參數進行序列化處理 這裏必須false
9         contentType: false, //必須
10        success: function (data) {
11            console.log(data.result)
12            // document.getElementById("result_image").innerHTML=data.result
13            $('#input_text').val(data.result)
14        },
15        error: function(xhr, status, error){
16            $('#response').text('Error: ' + error);
17        }
18    })
19 }
```

main.py

```
1  @app.route("/predict_text",methods=[ "GET",'POST'])
2  def predict_text():
3      sentence_e=request.form.get('sentence')
4
5      translator=googletrans.Translator()
6      result=translator.translate(sentence_e,dest='zh-tw')
7      translate=result.text
8      model_path = 'W11/bert_chinese_model'
9      tokenizer_name = 'bert-base-chinese'
10     model, tokenizer = load_model_and_tokenizer(model_path, tokenizer_name)
11
12     sentiment = predict_sentiment(translate, model, tokenizer)
13
14     return jsonify({"result":f"*原手語辨識 : {sentence_e}* <br> *中文翻譯 : {translate}* <br> *文本情感為 : {sentiment}*"})
15
16
17 app.run()
```

js(語意分析)

```
1  function post_data_text() {  
2  
3      var form_data = new FormData();  
4      form_data.append('sentence', $("#input_text").val());  
5  
6      $.ajax({  
7          url: "/predict_text",  
8          data: form_data,  
9          type: "post",  
10         dataType: "json",  
11         processData: false, //用於對data參數進行序列化處理 這裏必須false  
12         contentType: false, //必須  
13         success: function (data) {  
14             console.log(data.result)  
15             document.getElementById("result_text").innerHTML=data.result  
16         },  
17         error: function(xhr, status, error){  
18             $('#response').text('Error: ' + error);  
19         }  
20     })  
21 }
```

Html&Css

網頁主架構



```
1 <body>
2
3   <div class="header">
4     <h1>Sign Language</h1>
5   </div>
6
7   <div class="container">
8
9     <div class="left">
```

```
31
32 .left {
33   display: flex;
34   flex-direction: column;
35   justify-content: space-between;
36   background-color: #423a3a; /* 左半部分背景色 */
37   padding: 20px;
38 }
```

```
1 body, html {
2   margin: 0;
3   padding: 0;
4   font-family: Arial, sans-serif;
5   height: 100%;
6   background-color: #423a3a;
7 }
8
9 .header {
10   background-color: #ccc;
11   padding: 8px;
12 }
13 .header h1 {
14   text-align: center;
15   font-family: "IM Fell English", serif;
16   font-weight: bold;
17   font-style: italic;
18   font-size: 250%;
19   color: #423a3a;
20 }
21
22 .container {
23   display: flex;
24   height: calc(100vh - 160px);
25 }
26
```

```
.right {
  background-image: url("pic.png");
  background-size: cover;
  background-repeat: no-repeat;
}
```

Html

```
1 <div class="upload">
2     <h1>圖像預測</h1>
3     <form action="upload" method="post" enctype="multipart/form-data">
4         <input type="file" multiple="" name="file[]" class="span2">
5         <div class="span">
6             <input type="submit" value="上傳" style="width: 80px; height: 30px;font-size: 15px;">
7             <button type="button" onclick="post_data_image()" style="width: 80px; height: 30px;font-size: 15px;">手語翻譯</button>
8         </div>
9     </form>
10 </div>
```

```
1 <div class="predict">
2     <h1>語意辨識</h1>
3     <input type="text" id="input_text" style="width: 98%; height: 20px;font-size: 15px;">
4     <button onclick="post_data_text()" style="width: 80px; height: 30px; font-size: 15px;">確認</button>
5     <div id="result_text"></div>
6 </div>
7
8 </div>
9 <div class="right"></div>
10 </div>
11
12 </body>
```

```
.upload,
.predict {
  margin: 0 auto;
  width: 80%;
  flex: 1; /* 均分高度 */
  background: #ffffffbd; /* 背景色 */
  padding: 20px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
  border-radius: 10px;
  margin-bottom: 20px;
}
```

上下各半

```
.upload h1, .predict h1 {
  text-align: left;
  font-family: "PMingLiU", Arial, sans-serif;
  font-size: 30px;
  font-weight: bold;
  color: #423a3a;
}
```

標題

```
.form-inline {
  display: flex;
  align-items: center; /* 垂直居中 */
  justify-content: center; /* 水平居中 */
}
```

```
.form-inline .button-group {
  display: flex;
  align-items: center;
}
```

```
.form-inline input[type="file"] {
  margin-right: 10px; /* 文件輸入和按鈕間距 */
}
```

```
.form-inline .button-group input[type="submit"],
.form-inline .button-group button {
  margin-left: 10px; /* 上傳和手語間距 */
}
```

Css

按鈕設定

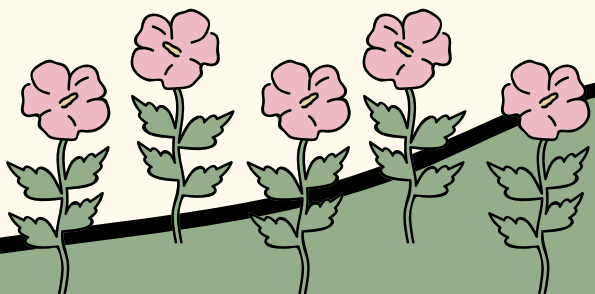
```
span:hover {
  padding: 10px 20px;
  background-color: #495057;
  color: black;
  border: 2px solid #ccc;
  border-radius: 5px;
  cursor: pointer;
  font-family: "PMingLiU", Arial, sans-serif;
}
```

懸浮按鈕

```
form {
  display: flex;
  flex-direction: column;
  align-items: center; /* 垂直居中 */
}
```

```
.span2 {
  width: 300px; /* 設置寬度 */
  height: 40px; /* 設置高度 */
  font-family: "PMingLiU", Arial, sans-serif;
  font-size: 20px;
}
```

```
input[type="file"] {
  margin-bottom: 30px;
}
```



組內分工

溫苾均	找資料集、概念發想、PPT製作、程式碼	33%
黃妤涵	找資料集、概念發想、PPT製作、程式碼	33%
李佩臻	找資料集、概念發想、PPT製作、程式碼	33%

